

Lecture 59 : Network Analysis using Wireshark

Introduction

In today's interconnected world, understanding the flow of data across networks is paramount, not just for network administrators but also for cybersecurity professionals and developers. This lecture introduces Wireshark, a powerful and widely-used tool for network analysis and packet sniffing. Often referred to as a "network protocol analyzer" or "sniffer," Wireshark allows users to capture and interactively browse the traffic running on a computer network.

The ability to inspect individual data packets provides invaluable insights into network behavior, application performance, and potential security vulnerabilities. From diagnosing network issues to investigating cyberattacks, Wireshark serves as an essential utility for anyone seeking to demystify the complex dance of data packets. This session will provide a comprehensive tour of Wireshark's features, its operational principles, and practical examples of its application in real-world scenarios.

Core Concepts

Term	Definition	Significance
Wireshark	An open-source, freely available network analysis tool used for capturing and interactively browsing network traffic.	It is a popular and powerful tool for profiling network activity, diagnosing issues, and analyzing security incidents.
Packet Sniffing	The process of analyzing network activity by capturing and inspecting individual data packets flowing through a network interface.	Essential for understanding network communication, troubleshooting, and identifying malicious activity or vulnerabilities.
Promiscuous Mode	A special operating mode for a Network Interface Card (NIC) that allows it to capture all packets on a network	Necessary for comprehensive network monitoring, as it enables the capture of traffic not explicitly addressed to the

Term	Definition	Significance
	segment, regardless of their intended destination.	sniffing machine. Requires root privileges.
Network Protocol Analyzer	A tool that captures and decodes network traffic, presenting it in a human-readable format according to various network protocols.	Provides detailed insight into the structure and content of network communications, aiding in debugging and security analysis.
Display Filters	Rules applied in Wireshark to selectively show only specific types of captured packets based on criteria like protocol, source/destination, or port numbers.	Crucial for managing and analyzing large volumes of captured data, allowing users to focus on relevant traffic.
Packet Summary Window	The top pane in Wireshark's main interface, displaying a list of all captured packets with high-level information like frame number, time, source/destination, protocol, and length.	Offers an overview of network activity and allows quick identification of packets of interest.
Protocol Window	The middle pane in Wireshark, showing the decoded details of a selected packet, breaking down its structure according to various protocol layers.	Provides in-depth information about the headers and fields of a specific packet, aiding in protocol analysis.
Data Window	The bottom pane in Wireshark, displaying the raw contents of a selected packet in hexadecimal and ASCII formats.	Allows examination of the actual data payload, useful for identifying cleartext information or specific data patterns.
Logging	The act of capturing and storing network data, which can be analyzed later.	Highly valuable for forensic investigations and evidence collection after a security

Term	Definition	Significance
		incident or for post-mortem analysis.
Root Privilege	Elevated administrative permissions required to initialize a network interface in promiscuous mode.	A security measure ensuring that only authorized users can perform deep packet inspection on a network.

Detailed Analysis

1. Understanding Wireshark's Core Functionality

Wireshark is a fundamental tool for network analysis, often referred to as a packet sniffer or network protocol analyzer. It is an open-source and freely available application, making it accessible for a wide range of users from students to seasoned professionals. Its primary function is to capture network packets flowing through a network interface and display them in a structured, human-readable format. This capability is crucial for profiling network traffic, which involves understanding what kind of data is moving across the network, by whom, and for what purpose.

One of the most critical aspects of Wireshark's operation is its ability to put a Network Interface Card (NIC) into **promiscuous mode**. When a NIC is in promiscuous mode, it captures not only the packets destined for its own machine but also all other packets traversing the local network segment. This comprehensive capture is essential for a complete network analysis. However, enabling promiscuous mode requires **root privileges** (or supervisory access) on the operating system, a security measure to prevent unauthorized packet interception. It's important to note that if a computer is connected via a Layer 2 switch, the switch might partition the LAN, potentially limiting the visibility of traffic on other segments. Therefore, the effectiveness of sniffing depends on the network environment and the sniffer's physical or logical placement.

The captured network data, often called "logging," is invaluable for **forensics and evidence collection**. In the event of a cyberattack, captured packets can be analyzed to reconstruct the attack sequence, identify the types of packet exchanges involved, and understand how the attack unfolded. This allows for post-incident analysis and the development of countermeasures. Wireshark also supports analyzing traffic generated by various applications, providing insights into their network behavior.

2. Wireshark User Interface and Packet Capture

Upon launching Wireshark, users are presented with an initial screen that allows them to select a network interface for capture. The main window, once a capture begins, is typically divided into three primary panes:

- **Packet Summary Window (Top Pane):** This pane lists all captured packets, with each row representing a single packet. Key columns include:
 - **No.:** Frame number.
 - **Time:** Timestamp of packet capture.
 - **Source:** Source IP address.
 - **Destination:** Destination IP address.
 - **Protocol:** The highest-level protocol identified (e.g., TCP, UDP, HTTP, ICMP, OSPF, STP).
 - **Length:** Size of the packet in bytes.
 - **Info:** A brief summary of the packet's content or purpose.
- **Protocol Window (Middle Pane):** When a packet is selected in the Packet Summary, this pane displays its decoded details, breaking down the packet into its various protocol layers (e.g., Ethernet, IP, TCP, HTTP headers). Users can expand or collapse these layers to view specific fields and their values.
- **Data Window (Bottom Pane):** This pane shows the raw content of the selected packet. On the left, the data is presented in hexadecimal format, including offsets. On the right, the same data is displayed in ASCII, allowing users to see any cleartext information within the packet.

Initiating a packet capture in Wireshark can be done in several ways:

1. **Double-clicking** directly on the desired network interface listed on the initial screen.
2. **Selecting** an interface with a single click and then clicking the "Capture" button.
3. Navigating to the **Capture menu** in the top menu bar and selecting the appropriate capture option.

3. Navigating and Filtering Captured Data

Once packets are being captured, the volume of data can be overwhelming.

Wireshark provides robust **display filters** to narrow down the displayed traffic to only what is relevant. The filter bar is located at the top of the main window. Users can type filter expressions (e.g., `tcp`, `ip`, `http`, `udp.port == 53`, `ip.addr == 192.168.1.1`) and press Enter or click the apply button. This will instantly update the Packet Summary window to show only packets matching the specified criteria.

For example, applying a filter like `tcp` will display only TCP packets, showing relevant information such as source/destination port numbers, flags (SYN, ACK, FIN), sequence numbers, acknowledgment numbers, window sizes, and packet lengths in the "Info" column. Similarly, an `ip` filter will show all IP packets, including those using protocols like NBNS, ICMP, or OSPF that run on top of IP, along with details like protocol version, source/destination IP addresses, header length, ID, flags, Time-to-Live (TTL), and header checksum.

Wireshark also offers a detailed list of predefined filters under the "Analyze" menu, such as `non-HTTP` and `non-SMTP`, `no ARP`, `no DNS`, `TCP only`, `UDP only`, or `IPv6 only`. These provide quick access to common filtering scenarios, often with explanations of their logic.

4. Exploring Wireshark's Menu Options

Wireshark's extensive functionality is organized through its menu bar:

- **File Menu:** Manages capture files.
- **Import:** `Open` (existing capture files), `Open Recent`, `Merge` (current capture with existing file), `Import from Hex Dump`.
- **Save:** `Save` (in Wireshark's native format), `Save As` (multiple formats available).
- **Export:** Export captured data into various formats or generate reports.
- **Print:** Print reports.
- **Close/Quit:** Close the current window or exit the application.
- **Edit Menu:** For manipulating displayed packets and preferences.
- **Find:** Search for packets based on hexadecimal strings.
- **Mark:** Selectively mark packets for later analysis.
- **Preferences:** Customize Wireshark's appearance and behavior, including font, color schemes, number of packets displayed, and visible columns.
- **View Menu:** Controls the display of the main window.
- **Window Layout:** Manage how the panes are arranged.
- **Expand/Collapse:** Show or hide details within the Protocol Window.
- **Coloring Rules:** Apply custom color coding to different types of packets for easier identification.
- **Go Menu:** Navigates through captured packets.
- `Next Packet`, `Previous Packet`, `First Packet`, `Last Packet`, `Go to Packet` (by number).

- Next Packet in Conversation , Previous Packet in Conversation .
- Browse History : Review previously viewed packets.
- **Capture Menu:** Manages the packet capture process.
- Start : Begin capturing packets.
- Stop : Halt the current capture.
- Options for configuring capture settings (e.g., interfaces, filters).
- **Analyze Menu:** Provides tools for in-depth analysis of captured data.
- **Display Filters:** As discussed, apply filters to the displayed packets.
- **Enabled Protocols:** Enable or disable decoding for specific protocols (e.g., uncheck ICMP if not interested in those packets).
- **Conversation Filter:** Quickly filter packets belonging to a specific conversation (e.g., IPv4 to see only IPv4 traffic).
- **Follow:** After selecting a packet, Follow TCP Stream or Follow HTTP Stream reconstructs the entire conversation between two endpoints, showing the application-layer data flow.
- **Statistics Menu:** Generates various statistical reports.
- **Capture File Properties:** Displays details about the capture host, hardware, OS, application, interface, and capture duration/packet count.
- **Protocol Hierarchy:** Shows a breakdown of protocols and their usage percentages, bytes, and packet counts (e.g., how many DNS packets were sent under UDP/IPv4).
- **Conversations:** Summarizes network conversations by protocol (e.g., TCP, UDP, IP), showing source/destination, packet counts, and byte counts.
- **Endpoints:** Lists all unique network endpoints (IP addresses, MAC addresses) and their associated traffic.
- **Telephony Menu:** Specialized tools for Voice over IP (VoIP) analysis.
- Allows tracking details of VoIP calls, including start/end times, initiator IPs, and packet exchanges for digitized voice data.
- **Wireless Menu:** Options for capturing data from wireless networks.
- Supports capturing from Wireless LAN (WLAN) and Bluetooth interfaces, providing specific tools for wireless traffic analysis.
- **Tools Menu:** Contains utilities for network configuration and security.

- **Scanning Rules:** Allows users to create rules for firewalls like IP tables (Linux), packet filters, or Windows Firewall, enabling custom filtering or blocking of traffic.

5. Practical Examples of Network Analysis

The lecture illustrates Wireshark's utility with several practical examples:

- **Analyzing a Google.com (HTTPS) Connection:**

- When connecting to `www.google.com`, Wireshark captures HTTP/TCP packets.
- Since it's HTTPS, the connection involves Transport Layer Security (TLS) exchanges. These exchanges include client and server "Hello" messages, key sharing, and cryptographic algorithm negotiation to establish a secure, encrypted communication channel.
- While the application data (e.g., web content) is encrypted, some initial commands or connection setup details might be visible in plaintext, allowing observation of the TLS handshake process.
- Using the "Follow HTTP Stream" or "Follow TCP Stream" option reveals the complete conversation, including HTTP commands like POST, HOST, and Keep-Alive.

- **Identifying Vulnerabilities on an Unsecured Website (vulnweb.com):**

- Capturing traffic while authenticating on a known vulnerable website (e.g., `vulnweb.com`) demonstrates a critical security risk.
- If the website does not use encryption (i.e., it's HTTP, not HTTPS), Wireshark can capture the username and password in **cleartext** within the packet data.
- The example shows a username "nptel" and password "password" being transmitted unencrypted, highlighting why such sites are deemed vulnerable to eavesdropping.

- **Verifying Security on a Secure Banking Site (SBI Net Banking):**

- In contrast, analyzing traffic during an SBI Net Banking session (a secure site) shows HTTP CONNECT requests on **port 443**, which is the standard port for HTTPS.
- This indicates a secure connection where all data, including sensitive banking information, is encrypted using TLS.
- Wireshark reveals extensive packet exchanges related to the TLS protocol, where keys are exchanged, and cryptographic protocols for encryption/

decryption are negotiated. This confirms that the communication is protected from simple packet sniffing.

These examples underscore how Wireshark can be used to both identify security flaws in unencrypted communications and verify the proper implementation of security protocols in secure applications.

Key Takeaways

- [] Wireshark is an essential, free, and open-source tool for capturing and analyzing network traffic.
- [] Packet sniffing requires placing the network interface in promiscuous mode, which necessitates root privileges.
- [] The Wireshark interface is divided into three main panes: Packet Summary, Protocol Window, and Data Window.
- [] Display filters are crucial for narrowing down and focusing on specific types of packets within a large capture.
- [] Wireshark's menus (File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools) offer extensive functionalities for managing, customizing, and analyzing network data.
- [] The "Follow TCP/HTTP Stream" feature allows reconstruction and viewing of entire application-layer conversations.
- [] Wireshark can expose unencrypted sensitive data (e.g., usernames/passwords) on insecure websites.
- [] Wireshark can verify the use of secure protocols like HTTPS and TLS by observing traffic on port 443 and TLS handshake exchanges.
- [] Captured network data is vital for network forensics and understanding the mechanics of cyberattacks.
- [] Practical experimentation with Wireshark is the best way to understand network processes and application behavior.

Conclusion

Wireshark stands out as an indispensable tool for anyone involved with computer networks. Its ability to capture, filter, and dissect network packets provides an unparalleled window into the intricate workings of data communication. From the high-level overview offered by the packet summary to the granular detail of hexadecimal data, Wireshark empowers users to diagnose performance issues, troubleshoot connectivity problems, and, critically, identify security vulnerabilities.

By understanding how to navigate its comprehensive menu options and apply effective display filters, users can transform raw network noise into actionable intelligence. The practical examples demonstrated, from analyzing secure web traffic to exposing cleartext credentials on vulnerable sites, highlight Wireshark's real-world impact. Mastering Wireshark is not just about learning a tool; it's about developing a deeper understanding of networking principles and the critical role they play in securing our digital landscape. This foundational knowledge is crucial for both proactive network management and reactive incident response, making Wireshark a cornerstone of modern network education and practice.